

ESCAPE

Emergency System for Calculating Adaptive Pedestrian Evacuations

User Manual

By:

Luis Moya

Department of Engineering, Pontifical Catholic University of Peru
(PUCP), Peru

Julio Ramírez

Department of Engineering, Pontifical Catholic University of Peru
(PUCP), Peru

Erick Mas

International Research Institute of Disaster Science
(IRIDeS), Tohoku University, Japan

Shunichi Koshimura

International Research Institute of Disaster Science
(IRIDeS), Tohoku University, Japan

**Funded by CONCYTEC-PROCIENCIA
(Contract Number PE501078853-2022)**

April 2025

Contents

I	References	5
0.1.	List of commands	7
0.1.1.	nodesFile	7
0.1.2.	linksFile	7
0.1.3.	populationsFile	7
0.1.4.	startTime	7
0.1.5.	process	8
0.1.6.	stopAt	8
0.1.7.	endTime	8
0.1.8.	stopSimulationAt	8
0.1.9.	endNumberSimulation	9
0.1.10.	meanRayleigh	9
0.1.11.	exploration	9
0.1.12.	graphicPrintout	9
0.1.13.	graphicPrintoutPeriod	9
0.1.14.	pedestrianCountPeriod	10
0.1.15.	computationContinued	10
0.1.16.	previousComputationFile	10
0.1.17.	totalEvacuatedCount	10
0.1.18.	pythonVersion	10
II	User manual	11
1	Introduction	13
1.1.	Presentation of the ESCAPE [®] software	13
1.2.	Main features	13
1.3.	Contact	13
1.4.	Requirements	14
2	Fundamentals	15
2.1.	Sarsa algorithm	15
2.2.	Sarsa parameters	15
3	Program structure and algorithm	17
3.1.	Program structure	17
3.2.	List of classes	17
3.3.	Software modules	17
3.3.1.	Node module (Intersections)	17

3.3.2.	Link module (Streets)	18
3.3.3.	nodeEvacuation module (Evacuation points)	19
3.3.4.	Time module	21
4	Installation	23
4.1.	Cloning the repository	23
4.2.	Loading the variables	23
4.3.	Compiling Sarsa	23
4.4.	Updating the code	23
5	Evacuation simulation	25
5.1.	Specification of the initial conditions	25
5.2.	Inputs and outputs	25
5.2.1.	Intersections file	25
5.2.2.	Streets file	26
5.2.3.	People file	26
5.2.4.	Control file	27
5.3.	Starting the program	27
5.4.	Example cases	28
5.4.1.	Grid case	28
5.4.2.	Camaná case	30
5.4.3.	Python case	31

Part I

References

0.1. List of commands

0.1.1. nodesFile

Attribute	Value	Detail
Type	String	Data type used for this command.
Default	nodes.csv	Default name of the file that contains the intersection information.
Description	Name of the intersections data file.	

0.1.2. linksFile

Attribute	Value	Description
Type	String	Data type used for this command.
Default	links.csv	Default name of the file that contains the street information.
Description	Name of the streets data file.	

0.1.3. populationsFile

Attribute	Value	Description
Type	String	Data type used for this command.
Default	population.csv	Default file that contains the population.
Description	Name of the people data file.	

0.1.4. startTime

Attribute	Value	Description
Type	Integer	Data type used for this command.
Default	0	Default value used to define the start time.
Description	Start time of the evacuation; it must be entered in seconds.	

0.1.5. process

Attribute	Value	Detail
Type	String	Data type used for this command.
Default	calibration	Default value used to define the simulation process.
Description	Indicates the simulation process.	
- Calibration	In this process the code needs to train the Sarsa algorithm so that it makes its own decisions. The calibration process allows it to explore and experiment with decisions in order to store the learning results; this process can be repeated according to the desired number of simulations.	
- Trained	In this process the Sarsa algorithm uses what it has learned. It only allows the previous learning to be used and, based on it, to model a simulation.	

0.1.6. stopAt

Attribute	Value	Detail
Type	String	Data type used for this command.
Default	endTime	Default option used to define the final evacuation time.
Description	Options for determining when to stop the evacuation. Only one is implemented: endTime .	

0.1.7. endTime

Attribute	Value	Detail
Type	Integer	Data type used for this command.
Description	Used to enter the last evacuation time up to which the simulation is to be run.	

0.1.8. stopSimulationAt

Attribute	Value	Detail
Type	String	Defines the data type as a character string.
Default	endNumberSimulation	The number of simulations at which to stop.
Description	Options for ending the simulations. Only one is implemented: endNumberSimulation .	

0.1.9. endNumberSimulation

Attribute	Value	Detail
Type	Integer	Data type used for this command.
Description	Number of simulations that will be run; the last iteration of the simulation is entered. Note that it is mandatory to set this value.	

0.1.10. meanRayleigh

Attribute	Value	Detail
Type	Integer	Data type used for this command.
Description	Mean of the Rayleigh distribution (the only adjustable parameter of the distribution). It is used to compute the scale factor needed to obtain a number from that distribution. The value must be entered in seconds. This command is mandatory.	

0.1.11. exploration

Attribute	Value	Detail
Type	Double	Data type used for this command.
Default	close to 0.2	Default value used for the exploration percentage at the end of the simulation.
Description	Final exploration value. As the simulations proceed, the exploration rate changes until it reaches the value entered.	

0.1.12. graphicPrintout

Attribute	Value	Detail
Type	Logical	Data type used for this command.
Default	Yes	Enabled for exporting variables.
Description	Used to export the position, velocity and number of evacuees variables.	

0.1.13. graphicPrintoutPeriod

Attribute	Value	Detail
Type	Integer	Data type used for this command.
Default	1	Default value associated with the attribute.
Description	Time interval at which the variables are exported, i.e. the export interval. It can be greater than 1. It must be entered in seconds.	

0.1.14. pedestrianCountPeriod

Attribute	Value	Detail
Type	Integer	Data type used for this command.
Default	1	Default value associated with the attribute.
Description	Determines how often the count of people on the streets is calculated.	

0.1.15. computationContinued

Attribute	Value	Detail
Type	Logical	Data type that indicates true or false.
Default	No	Default value assigned to the attribute.
Description	Determines whether the calculations will be independent of a previous result.	

0.1.16. previousComputationFile

Attribute	Value	Detail
Type	String	Defines the data type as a character string.
Description	Name of the file, located in the <code>stateMatrices</code> folder, that contains the results of a previous simulation of the same city. That simulation provides experienced state values for the new simulation. It is required when <code>computationContinued</code> is enabled and when <code>process</code> is trained.	

0.1.17. totalEvacuatedCount

Attribute	Value	Detail
Type	Logical	Data type that indicates true or false.
Default	Yes	Default value assigned to the attribute.
Description	Allows exporting a file with the total number of people evacuated during the evacuation time of the trained process.	

0.1.18. pythonVersion

Attribute	Value	Detail
Type	Logical	Data type that indicates true or false.
Default	No	Default value assigned to the attribute.
Description	Allows using learning data obtained from the Python version.	

Part II

User manual

Chapter 1

Introduction

1.1. Presentation of the ESCAPE[®] software

The ESCAPE[®] code uses the Sarsa algorithm to solve speed–time equations in order to evaluate escape routes when a tsunami reaches the coasts of cities.

The main results of the evacuation of each pedestrian are the following:

- Evacuation time
- Position
- Velocity

1.2. Main features

The main features of the software are the following:

- **Software:** Developed in C++, a low-level programming language that guarantees more efficient and faster performance.
- **Evacuations:** Facilitates the planning and execution of urban evacuations.
- **Evacuation time:** Makes it possible to accurately calculate the total time required to evacuate the inhabitants of a city.
- **Destination points:** Provides tools to identify and establish strategic evacuation points in urban areas.

1.3. Contact

If you have any questions while using the software, you can contact us by sending an email to receive technical assistance.

Email addresses:

lmoya@pucp.edu.pe

julio.ramirez@pucp.edu.pe

1.4. Requirements

- Language: C++ (C++17 support is required)
- Compiler: GCC 9.4 or higher / Clang 10.0 or higher
- Build tools:
 - GNU Make 4.2.1 (Makefile)
 - Operating system: Ubuntu 20.04.6 LTS or macOS
- Optional tools:
 - gnuplot: used by the program to write the evacuation plots (without it the plots are skipped and the simulation still runs)
 - Python 3 with matplotlib, progressbar2 and screeninfo, and ffmpeg: used by the post-processing scripts in each tutorial

Chapter 2

Fundamentals

The Sarsa algorithm (State–Action–Reward–State–Action) was proposed by Rumery and Niranjana under the name “Modified Connectionist Q-Learning” (MCQ-L). It was Rich Sutton who gave it the name Sarsa.

The name Sarsa comes from the following chain: the main function for updating the Q value depends on the current state of the agent “ S_1 ”, the action chosen by the agent “ A_1 ”, the reward “ R_2 ” that the agent obtains for choosing this action, the state “ S_2 ” that the agent enters after performing that action, and, finally, the next action “ A_2 ” that the agent chooses in its new state. The result is the acronym of the quintuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$.

2.1. Sarsa algorithm

The Sarsa algorithm is the following:

$$Q_{\text{new}}(S_t, A_t) \leftarrow (1 - \alpha) Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})] \quad (2.1)$$

A Sarsa agent is known as a learning algorithm because it interacts with the environment and updates the policy based on the actions taken. The Q value of a state–action pair is updated through an error and is adjusted with a learning rate “ α ”.

The Q values represent the possible reward received at the next time step for taking action a in state s , plus the discounted future reward received from the next state–action observation.

An advantage of Sarsa is that it learns by itself the Q values associated with adopting the policy it follows.

2.2. Sarsa parameters

The parameters used with Sarsa are shown below:

- **Learning rate (α):** Determines to what extent newly acquired information overrides old information. A factor of 0 makes the agent learn nothing, while a factor of 1 makes the agent consider only the most recent information.
- **Discount factor (γ):** Determines the importance of future rewards. A discount factor of 0 makes the agent opportunistic or myopic, considering only current rewards. On the other hand, a factor close to 1 encourages the agent to seek high

long-term rewards. If the discount factor is equal to or greater than 1, the Q value may diverge.

- **Initial conditions** ($Q(S_0, A_0)$): Since Sarsa is an iterative algorithm, it assumes an initial condition before the first update. A high or infinite initial value, also known as “optimistic initial conditions”, can encourage exploration. The initial update rule assigns high values to all actions, increasing their probability of being chosen. The first time an action is taken, the reward “ r ” resets the initial conditions and is used to set the value of Q , allowing immediate learning in the case of deterministic rewards.

Chapter 3

Program structure and algorithm

3.1. Program structure

The ESCAPE[®] software needs the following input information: the streets (link), the intersections of the streets (node) and the people (pedestrian), each with its identifier. Once all the information has been entered, the data are processed.

After the data have been processed, the outputs are tables and/or figures showing the number of evacuees vs. time, the number of evacuees vs. evacuation points, and the total number of evacuees vs. simulation number.

3.2. List of classes

Class	Description
io	File input and output.
dictionary	Command dictionary.
node	Definition of intersections in a city.
link	Definition of streets in a city.
subLink	Definition of a subsection of a street.
pedestrian	Definition of pedestrians.
stateMatrix	State matrix structure.
sarsa	Computation of the Sarsa algorithm.

Table 3.1: Names of the classes used and implemented in the model.

3.3. Software modules

3.3.1. Node module (Intersections)

The Node module has the following members:

- **idNode**: Unique identifiers of each intersection.
- **coordinate**: Contains the location vector of the intersection.
- **linkConnectionsPtr**: A vector of the interconnected streets.

- `experiencedStateMatrixsPtr`: A vector of the experienced states.

The members `idNode` and `coordinate` are identified with the intersections file; whereas `linkConnectionsPtr` and `experiencedStateMatrixsPtr` are members that are obtained as the software processes the data.

Assignment of intersection values

To enter the intersection values, a csv file is needed that contains the intersection identification values, the position in x, the position in y and the node type (the complete format is described in Section 5.2.1.).

The control file `controlDict` has the command `nodesFile` to specify the name of the file; by default this is `nodes.csv`.

An example of changing the name of the intersections file is shown below:

```
1 nodesFile: intersectionsLima.csv
```

3.3.2. Link module (Streets)

The Link module has the following members:

- `idLink`: Unique identifier of a street.
- `node1Ptr`: Pointer to the first intersection of the street.
- `node2Ptr`: Pointer to the second intersection of the same street.
- `length`: Length of the street.
- `width`: Width of the street.
- `linkOrientation`: Orientation of the street.
- `subdivisionWidth`: Width of each subdivision of the street.
- `subdivisionCount`: Number of subdivisions of the street, used for the density calculation.
- `densityLevel`: Density of the street, calculated as the maximum density among its subdivisions.
- `subdivisions`: Vector of the `subLink` class, which represents the divisions of the street.

Assignment of street values

To enter the street values, a file is needed in which the following are given: an identifier number for the street, the two start and end points of the street, and the length and width of the street. The control file `controlDict` has the command `linksFile` to specify the name of the file, which by default is `links.csv`.

An example of changing the name of the streets file is shown below:

```
1 linksFile: streetsLima.csv
```

Options for subdividing a street

There are two options for subdividing a street:

1. `subdivisionWidth`
2. `subdivisionCount`

By default the `subdivisionWidth` option is enabled. This option divides the street according to a fixed length for each subdivision, and from it the number of subdivisions is calculated. If the number of subdivisions is not an integer, it is rounded and a new width is calculated again, which is very close to the previous one; this is done because the number of subdivisions cannot be a decimal value.

On the other hand, the second option allows entering a number of divisions for the street, from which a single width for each subdivision will be calculated.

An example of the first option, which appears by default, for the subdivision of a street is shown below:

```
1 subdivisionOption: subdivisionWidth
```

Subdivision width

To use this option it is not mandatory to choose `subdivisionWidth` in the `subdivisionOption` command, because that option is already the default.

It allows setting the value of the length of the subdivisions. The values can be integers or decimals, but not fractions.

An example for the subdivision of a street is shown below:

```
1 subdivisionWidth: 12.5
```

Number of subdivisions

To use this option it is necessary to choose `subdivisionCount` in the `subdivisionOption` command.

The `subdivisionCount` command allows dividing the street into an integer number of subdivisions.

An example of using the command is shown below:

```
1 subdivisionCount: 20
```

3.3.3. nodeEvacuation module (Evacuation points)

The `nodeEvacuation` module has the following members:

- **maxEvacuatedPeople:** The number of people who can evacuate at this site. By default, this option is disabled, so each evacuation point has infinite capacity. To enable this option and limit the capacity, the `intersections` file must be modified by entering the value 2, followed by the maximum number of evacuees for that point.

Printing the Evacuees vs. Time figure for different simulations

It shows a line with the information on the total number of people evacuated over the whole evacuation time for certain simulations. The desired simulation number is added with the command `evacuatedVsTimeAt`, showing how the learning of the algorithm evolves. This information is exported in png format.

An example of enabling the figure is shown below:

```
1 figureEvacuatedVsTime: yes
```

Printing the Evacuees vs. Time table for different simulations

It prints a table with the values of the total number of evacuees over the whole evacuation time for certain simulations. The desired simulation number is added with the command `evacuatedVsTimeAt`, showing how the learning of the algorithm evolves. This information is exported in csv format.

An example of enabling the table is shown below:

```
1 tableEvacuatedVsTime: yes
```

Evacuation vs. Time for specific simulations

The printing of the table and the figure of the total number of evacuees vs. time uses simulation numbers 1, 2 and 3 by default. To change these values, the command `evacuatedVsTimeAt` can be used, to which integer values separated by commas must be assigned.

An example of assigning simulations is shown below:

```
1 evacuatedVsTimeAt 1, 5, 8;
```

Printing the Total evacuees vs. Simulation figure

It shows a single-line plot with the information on the total number of people evacuated over the course of the simulations, showing how the learning of the algorithm evolves. This information is exported in png format.

An example of enabling the figure is shown below:

```
1 figureTotalEvacuatedVsSimulation: yes
```

Printing the Total evacuees vs. Simulation table

It shows a table with the values of the total number of evacuees over the course of the simulations, showing how the learning of the algorithm evolves. This information is exported in csv format.

An example of enabling the table is shown below:

```
1 tableTotalEvacuatedVsSimulation: yes
```

Simulation period of the Total evacuees vs. Simulation output

The printing of the table and the figure of the total number of evacuees vs. simulation is done every simulation, which is a default value. To change this value, the command `totalEvacuatedVsSimulationPeriod` can be used, to which the value of how often simulation values are wanted must be assigned.

An example of assigning the simulation period is shown below:

```
1 totalEvacuatedVsSimulationPeriod 5;
```

3.3.4. Time module

The Time module has the following members:

- `timeValue`: Value of the simulated time, which increases according to `deltaT`.
- `deltaT`: Time step value. By default, it is 1 second.
- `startNumberSimulation`: Value of the initial simulation. By default, it is 0.
- `endNumberSimulation`: Value of the last simulation. It must be entered in the `controlDict` file.
- `graphicPrintoutPeriod`: Value that indicates how often the position, velocity and number of evacuated people variables should be exported.
- `pedestrianCountPeriod`: Value that indicates how often the density should be calculated.

Evacuation time

This value is used to set, as a mandatory entry, the evacuation time in seconds.

An example is shown below:

```
1 endTime 800;
```

Setting the number of simulations

This value is used to set, as a mandatory entry, the number of simulations. That is, each simulation carries out one evacuation. Only integer values greater than 0 are accepted.

An example is shown below:

```
1 endNumberSimulation 7000;
```

Variable printing period

This command makes it possible to print the variables at regular intervals, which are given in seconds. For example, if the command has the value 5 and the command `endTime` is 50, then the variables will be printed in the `data` folder at the following times: 0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50 by default.

An example is shown below:

```
1 graphicPrintoutPeriod 5;
```

Density calculation period

This command makes it possible to count the people in each subdivision of the street in order to then calculate the density of each subdivision and obtain the maximum value, which is assigned to the street as its density level. For example, if the command has the value 6 and the command `endTime` is 50, then the density will be calculated at times 0, 6, 18, 24, 30, 36, 42 and 48.

An example is shown below:

```
1 pedestrianCountPeriod 6;
```

Chapter 4

Installation

This guide details the installation, simulation and post-processing process for some application cases of the Sarsa code. The `tutorials` directory contains demonstration cases of the use of the algorithm and its utilities in different scenarios. Before trying to run the tutorials, the user must make sure that Sarsa is correctly installed.

4.1. Cloning the repository

To clone the repository, run the following:

```
1 > git clone https://github.com/juliocesar-ramirez/cplus-version.git
```

4.2. Loading the variables

The Sarsa environment variables are defined in the `bashrc` file, located in the root directory of the repository.

```
1 > cd cplus-version
2 > source bashrc
```

4.3. Compiling Sarsa

With the environment loaded, compilation of Sarsa starts by running the script in the `makefile`.

```
1 > make
```

4.4. Updating the code

The source code is updated regularly. Therefore, these modifications and new files can be downloaded using git.

```
1 > git pull
```


Chapter 5

Evacuation simulation

5.1. Specification of the initial conditions

There are two initial conditions for starting the evacuation simulation:

- **Positions of the people at the start of the evacuation:** The positions where the people are located cannot be modified through the nodes file.
- **Departure time of the people at the start of the evacuation:** The time at which the people will begin to evacuate is an initial condition based on the Rayleigh distribution, whose parameter can be adjusted by the user.

5.2. Inputs and outputs

A list of files is used by the ESCAPE[®] program; some of them are inputs and others are outputs. Likewise, some of them are optional. The input files are the following:

- The streets file (mandatory) contains the list of streets of the city and their location.
- The intersections file (mandatory) contains information about the intersections of the streets.
- The people file (mandatory) contains information about the pedestrians and their starting position.

5.2.1. Intersections file

The user can change the name of the file by modifying the `nodesFile` command. The file contains the following:

- Id of the intersection, a unique number that cannot be equal to that of another intersection.
- x coordinate
- y coordinate
- Type of node: 0 for a regular intersection, 1 for an evacuation point, and 2 for an evacuation point with limited capacity, in which case the maximum number of evacuees for that point follows.

- An integer that must be present at the end of each line (it is read by the program but not used).

```
# Example of the nodes.csv file
# id,x,y,type,unused
0,0,0,0,1
1,20,0,0,1
2,40,0,0,1
3,60,0,0,1
4,80,0,0,1
```

The order is not important. Lines that begin with `#` are taken as comments. The number of points is free and does not necessarily have to be the same as in the example.

5.2.2. Streets file

The user can change the name of the file by modifying the `linksFile` command. The file contains the following:

- Id of the street, a unique number that cannot be equal to that of another street.
- Id of the start intersection.
- Id of the exit intersection.
- Length of the street.
- Width of the street.

```
# Example of the links.csv file
# id,node1,node2,length,width
0,0,1,20,3
1,1,2,20,3
2,2,3,20,3
3,3,4,20,3
4,4,5,20,3
```

The order is not important. Lines that begin with `#` are taken as comments. The number of streets is free and does not necessarily have to be the same as in the example.

5.2.3. People file

The user can change the name of the file by modifying the `populationsFile` command. The file contains the following:

- Age of the person
- Gender of the person, 1 if male.
- Category HHType
- Category HHID
- Starting intersection of the person.

```
# Example of the population.csv file
# age,gender,HHType,HHID,closeNode
18,1,0,0,26
18,1,0,0,22
18,1,0,0,20
18,1,0,0,6
```

The order is not important. Lines that begin with `#` are taken as comments. The number of people is free and does not necessarily have to be the same as in the example.

5.2.4. Control file

The user must see this file, called `controlDict`, located in the `system` directory. Import of mandatory and optional input data, time control, export.

The file represents a control panel of the ESCAPE[®] code. It contains a number of commands whose values can be assigned or used with their default values. All the commands are defined in the list of commands in Part I.

```
1 nodesFile          nodes.csv;
2
3 linksFile          links.csv;
4
5 populationsFile    population.csv;
6
7 stopAt             endTime;
8
9 endTime            800;
10
11 deltaT             1;
12
13 meanRayleigh       420;
14
15 graphicPrintoutPeriod 1;
16
17 computationContinued no;
18
19 previousComputationFile sim_000006000.csv;
20
21 stopSimulationAt    endNumberSimulation;
22
23 endNumberSimulation 6003;
```

Lines that begin with `#` are taken as comments.

5.3. Starting the program

The calculations are launched from the terminal using the command `sarsa`. The command triggers the execution of a main script containing all the modules, functions and variables of the code. The syntax of the command is the following:

The program is invoked by typing the command `Sarsa` in the terminal. The command runs a binary file stored in the `bin` folder, which is a binary of all the scripts in the `src` folder.

The syntax of the command is the following:

```
1 > sarsa
```

5.4. Example cases

ESCAPE[®] includes tutorial cases to show the functionalities of the code. They are available in the `tutorials` folder (`grid`, `camana` and `diagonal`). The grid and Camaná cases are described below, followed by the use of data from the Python version of the code.

5.4.1. Grid case

This tutorial (folder `tutorials/grid`) will describe how to pre-process and post-process a case with an ideal city in the shape of a rectangle. The geometry is shown in Figure 5.1, where the lines are the streets. The people are initially positioned at the intersection points of the streets. The initial movement speed of the people will be 1 m/s.

This example will show the most optimal evacuation route when the people leave from the same starting point to their evacuation point. The configuration of the city has streets with a characteristic pattern, which are junctions of 4 streets that form a rectangle, with a total of 25 blocks of the same shape, as shown in Figure 5.1.

Likewise, a street has a length of 20 meters and a width of 3 meters. This configuration has a single evacuation point located at position (100, 100).

The user must enter certain information in the `controlDict`:

- `endTime`: Final evacuation time with the `endTime` command.
- `endNumberSimulation`: The final number of simulations with the `endNumberSimulation` command.
- `process`: Type of process with the `process` command.
- `meanRayleigh`: The mean value for the Rayleigh distribution.
- `populationsFile`: The name of the population file.

The input data about time control, reading, printing and solution values will be read from the `controlDict` file. The user must modify it beforehand.

For the present case, the `controlDict` file with the values of the mandatory commands is shown below.

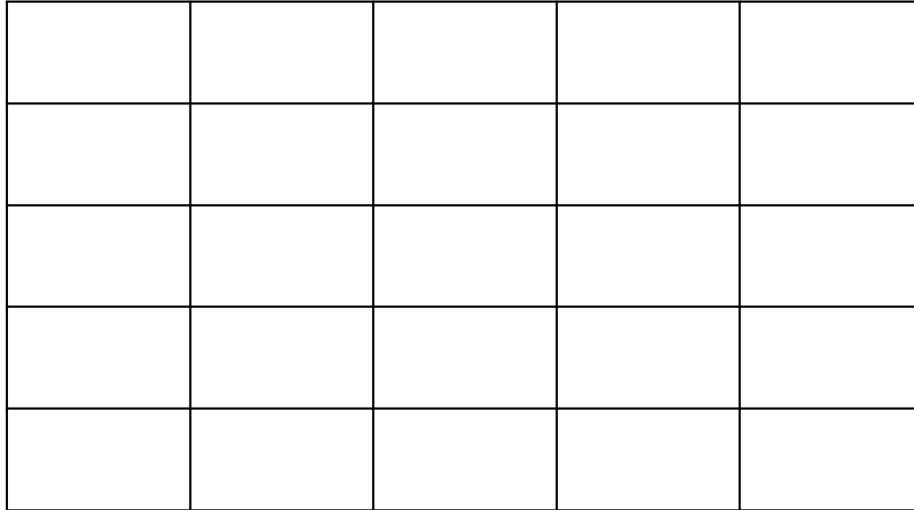


Figure 5.1: Plan view of the city of the grid case.

```

1 populationsFile  populationAtPoint.csv;
2
3 process calibration;
4
5 endTime          1800;
6
7 meanRayleigh     420;
8
9 endNumberSimulation 1000;

```

There are commands whose value does not need to be specified, because their default value will be used, which is listed in the commands section.

Once the `controlDict` has been defined, it is recommended to clean the working folder; for this there is an external script named `Allclean`.

The program is run with the command `sarsa`. The information printed in the terminal during a run looks as follows.

```

1 ***** Simu: 410 *****
2 epsilon greedy - exploration: 0.379363
3 survived pedestrian: 6195
4 Duration: 0 min / 3 s / 3018 ms
5
6 ***** Simu: 411 *****
7 epsilon greedy - exploration: 0.378788
8 survived pedestrian: 6216
9 Duration: 0 min / 3 s / 3040 ms
10
11 ***** Simu: 412 *****
12 epsilon greedy - exploration: 0.378215
13 survived pedestrian: 6225
14 Duration: 0 min / 2 s / 2899 ms
15

```

```

16 ***** Simu: 413 *****
17 epsilon greedy - exploration: 0.377644
18 survived pedestrian: 6231
19 Duration: 0 min / 2 s / 2972 ms
20
21 ***** Simu: 414 *****
22 epsilon greedy - exploration: 0.377074
23 survived pedestrian: 6220
24 Duration: 0 min / 3 s / 3005 ms

```

Upon completion, the results of each variable, tables with information on the number of evacuees and validation images can be obtained.

Finally, the export of the variables allows the user to post-process the information in any other program. Example post-processing scripts written in Python are provided in the `postprocessing` folder of each tutorial.

5.4.2. Camaná case

This example (folder `tutorials/camana`) will show the evacuation of people in the district of Camaná with several evacuation point options, all far from the beach, as shown in Figure 5.2. The configuration is an idealization based on the street layout of the district of Camaná, with a maximum street length of 600 meters and a width of 3 meters.

The configuration of this case is the following:



Figure 5.2: Plan view of the city of the Camaná case.

- `endTime`: Final evacuation time: 1800 sec.
- `endNumberSimulation`: Final number of simulations: 1000.
- `meanRayleigh`: The mean value for the Rayleigh distribution. Average departure time of a person of 420 sec.

The `controlDict` file is shown below.

```
1 populationsFile  population.csv;  
2  
3 process  calibration;  
4  
5 endTime        1800;  
6  
7 meanRayleigh    420;  
8  
9 endNumberSimulation 1000;
```

Finally, in the `postprocessing` folder there are files to perform visualizations of the data externally to the main program. The file `snapshot.py` provides images at each time step. The file `zoom.py` makes it possible to create images of a wide area to observe the evacuation movement in detail.

5.4.3. Python case

If you want to use `stateMatrix` data from previous simulations, but from the Python version of the same code, the `pythonVersion` command must be enabled. This option must be enabled because the order of the street connections at an intersection is different between the different languages.